

For your objective of **Cryptographic Authentication and Optimization of Security Overhead in Blockchain-Assisted Multipath ICN**, the methodology should focus on **lightweight security mechanisms** that preserve content integrity and authenticity while minimizing computational, communication, and blockchain overhead.

The following methodology is suitable for an SCI Q1 journal paper.

E. Blockchain-Assisted Cryptographic Authentication and Security Overhead Optimization

The proposed framework integrates lightweight cryptographic mechanisms with a permissioned blockchain architecture to ensure secure, efficient, and trustworthy content dissemination in multipath ICN environments. The methodology aims to authenticate content producers, verify chunk integrity, and minimize security-related computational and communication overhead.

The complete process consists of five phases:

1. Producer authentication and registration;
 2. Secure chunk generation and encryption;
 3. Blockchain-based hash registration;
 4. Consumer-side cryptographic verification; and
 5. Security overhead optimization.
-

1. Producer Authentication and Registration

Before publishing any content, each producer must be authenticated within the blockchain network.

The producer registration process includes:

- Generation of a unique producer identifier PID_p ;
- Creation of an Elliptic Curve Cryptography (ECC) public-private key pair;
- Storage of the producer's public key within the blockchain ledger.

ECC is selected because it provides equivalent security to RSA while significantly reducing computational complexity and key size, thereby minimizing security overhead.

The producer identity is represented as:

$$PID_p = H(PK_p || T_p)$$

where:

- PK_p denotes the producer public key;
- T_p represents the registration timestamp;
- $H(\cdot)$ denotes the SHA-256 hash function.

The blockchain stores only lightweight authentication metadata rather than complete content information.

2. Secure Chunk Generation and Cryptographic Authentication

After authentication, the producer divides the content object into fixed-size chunks.

Let:

$$N = \{C_1, C_2, \dots, C_K\}$$

denote the content object containing K chunks.

To reduce encryption overhead, a symmetric AES-256 session key is generated:

$$SK = \text{GenerateSessionKey}()$$

Each chunk is encrypted using:

$$E_i = AES_{SK}(C_i)$$

where:

- E_i denotes the encrypted chunk.

The integrity of the encrypted chunk is then computed using SHA-256:

$$H_i = SHA256(E_i)$$

The producer subsequently generates a digital signature using ECC:

$$Sig_i = ECC_{Sign}(H_i, SK_{priv})$$

where:

- SK_{priv} denotes the producer's private signing key.

This hybrid cryptographic mechanism combines:

- AES-256 for efficient data encryption;
- SHA-256 for integrity verification;
- ECC for lightweight authentication.

Compared with conventional RSA-based approaches, ECC substantially reduces cryptographic processing overhead.

3. Blockchain-Based Metadata Registration

To minimize blockchain storage requirements, only essential metadata are recorded on the ledger.

The blockchain registration record for chunk i is defined as:

$$BR_i = \{CID_i, H_i, PID_p, T_i\}$$

where:

- CID_i denotes the chunk identifier;
- H_i represents the chunk hash;
- PID_p denotes the producer identifier;
- T_i denotes the creation timestamp.

The encrypted chunk itself is never stored on the blockchain.

This significantly reduces:

- storage overhead;
- transaction latency;
- consensus processing costs.

The blockchain therefore functions solely as a decentralized trust repository for cryptographic verification.

4. Consumer-Side Cryptographic Verification

When content is received through multiple transmission paths, the consumer verifies each chunk independently.

For every received encrypted chunk E_i , the consumer computes:

$$H'_i = SHA256(E_i)$$

The corresponding blockchain record is then retrieved:

$$H_i^{BC} = Lookup(CID_i)$$

Content authenticity is verified as:

$$V_i = \begin{cases} 1, & H'_i = H_i^{BC} \\ 0, & otherwise \end{cases}$$

If verification succeeds, the producer signature is validated using the stored public key:

$$ECC_{Verify}(Sig_i, H_i, PK_p) = True$$

Only authenticated chunks are accepted and reassembled.

This mechanism effectively prevents:

- cache poisoning attacks;
- content tampering;
- malicious producer impersonation;
- replay attacks.

5. Security Overhead Optimization

To reduce the performance impact of blockchain-based security mechanisms, the proposed framework introduces four optimization strategies.

A. Hybrid Cryptographic Architecture

Instead of using asymmetric encryption for entire content objects, the framework employs:

- AES-256 for bulk data encryption;
- ECC for authentication;
- SHA-256 for integrity verification.

The total cryptographic cost becomes:

$$C_{crypto} = C_{AES} + C_{SHA} + C_{ECC}$$

which is substantially lower than:

$$C_{RSA}$$

for large-scale content dissemination.

B. Lightweight Blockchain Storage

Only hashes and metadata are registered:

$$O_{BC} = |CID| + |H| + |PID| + |T|$$

rather than:

$$O_{BC} = |Content|$$

This minimizes:

- ledger growth;
 - consensus delay;
 - transaction costs.
-

C. Batch Blockchain Transactions

Multiple chunk hashes are aggregated into a Merkle tree:

$$MR = \text{Merkle}(H_1, H_2, \dots, H_K)$$

Only the Merkle root is stored on the blockchain:

$$\text{Store}(MR)$$

rather than registering each chunk individually.

The blockchain transaction overhead becomes:

$$O(1)$$

instead of:

$$O(K)$$

This optimization significantly improves scalability.

D. Adaptive Security Processing

The framework dynamically adjusts cryptographic operations according to content sensitivity.

Critical content:

- AES-256 encryption;
- ECC signatures;
- Full blockchain registration.

Non-critical content:

- Hash-based authentication only;
- Batch blockchain updates.

The security overhead can therefore be expressed as:

$$SO = \alpha C_{crypto} + \beta C_{comm} + \gamma C_{blockchain}$$

where:

- C_{crypto} denotes cryptographic computation cost;
- C_{comm} denotes communication overhead;
- $C_{blockchain}$ denotes blockchain transaction cost;
- α, β, γ are weighting parameters.

The optimization objective is:

This objective ensures strong content authentication while maintaining efficient multipath dissemination performance.

Recommended Technologies for Implementation

Component	Recommended Technology
Blockchain Platform	Hyperledger Fabric (preferred) or Ganache for prototyping
Symmetric Encryption	AES-256
Hash Function	SHA-256
Digital Signature	ECC (secp256r1)
Merkle Tree Construction	SHA-256 Merkle Root
Smart Contracts	Chaincode (Fabric) or Solidity (Ethereum)
ICN Simulator	ndnSIM (NS-3)

This methodology directly addresses your objective of **Cryptographic Authentication and Optimization of Security Overhead** and can serve as a complete methodology subsection in a blockchain-assisted secure multipath ICN journal paper.